



SyncUp

Real-Time Collaboration Platform

Project implementation report

A mobile-first team communication product for workspaces, channels, direct conversations, file sharing, and persistent message history.

Report status	Implementation and deployment review
Prepared	25 September 2026
Frontend	Flutter for Android, iOS, and web-capable builds
Backend	Node.js, TypeScript, Express, Socket.IO
Live API	https://syncup-api-production.up.railway.app
Health check	GET /api/v1/health -> HTTP 200, {status: ok}

DEPLOYMENT VERIFIED

The Railway deployment was checked while preparing this report. The latest deployment completed successfully and the public health endpoint returned HTTP 200. Secrets are intentionally excluded from this document.

1. Executive summary

SyncUp is implemented as a WhatsApp-inspired collaboration application organized around workspaces. Users authenticate with email and password, create or join multiple workspaces using rotating join codes, discover channels and members, start direct conversations, exchange messages in real time, and retrieve persisted history after reconnecting or restarting the app.

The client is built with Flutter and Riverpod. The server is a TypeScript Express service with Socket.IO for authenticated real-time events, MongoDB/Mongoose for durable data, Redis adapter support for horizontal socket broadcasting, and Cloudinary for validated attachments. Railway hosts the API with MongoDB Atlas and Cloudinary configured through environment variables.

Implemented scope

- Email/password registration, login, current-user lookup, logout, and rotating refresh tokens.
- Multiple workspaces per user, generated join codes, membership roles, member listing, and workspace-aware naming.
- Public and private channels, channel membership controls, archive/update operations, and conversation details.
- Direct messages by workspace member identity, with one unique conversation per participant pair and workspace.
- Persistent messages with cursor-based history, client IDs for optimistic reconciliation, quoted replies, edit, and soft delete.
- Socket.IO typing indicators, online presence, delivery/read receipts, authenticated room access, and operation errors.
- Cloudinary-backed image/PDF/office-document uploads up to 10 MB with chat image previews.
- Light/dark system theme, loading/empty/offline/reconnecting/retry states, and a SyncUp-branded interface.

Release boundary

Push notifications, voice/video calls, reactions, message search, pinned messages, and client-side end-to-end encryption are outside the v1 scope. Transport security is provided by HTTPS/WSS and data protection is delegated to configured database and storage providers.

2. Technology and library stack

Layer	Technology / libraries	Purpose
Mobile client	Flutter, Dart, Material 3	Cross-platform Android/iOS UI and responsive web-capable build.
State and navigation	flutter_riverpod, go_router	Reactive auth/workspace/conversation state and route guards.
Networking	Dio, socket_io_client	REST calls, refresh handling, authenticated Socket.IO session.
Device services	flutter_secure_storage, file_picker, intl	Token storage, attachment selection, date/time presentation.
API	Node.js, TypeScript, Express	Layered HTTP API under /api/v1 with typed build output.
Realtime	Socket.IO, @socket.io/redis-adapter, ioredis	Authenticated rooms, presence, typing, receipts, and multi-instance fan-out.
Persistence	MongoDB Atlas, Mongoose	Users, sessions, workspaces, memberships, conversations, messages, read state.

Layer	Technology / libraries	Purpose
Files	Cloudinary, multer	Validated multipart uploads and durable CDN URLs.
Security	bcryptjs, jsonwebtoken, helmet, cors, express-rate-limit, zod	Password hashing, JWTs, headers, origin policy, rate limiting, validation.
Quality	Vitest, Supertest, flutter_test, flutter_lints	Backend API tests, compile checks, widget/unit tests, static analysis.
Operations	Docker Compose, Railway, Dockerfile	Local MongoDB/Redis and public always-on API deployment.

3. Architecture and working flow

High-level architecture

Component	Responsibility	Connection
Flutter app	Screens, providers, optimistic message state, secure token lifecycle, attachment preview.	HTTPS REST + WSS Socket.IO
Express API	Authentication, authorization, workspace/channel/conversation operations, history, uploads.	REST /api/v1
Socket.IO layer	Room membership, message acknowledgements, typing, presence, read receipts.	Authenticated WebSocket upgrade
MongoDB Atlas	Durable application data and reverse chronological message history.	Mongoose connection
Redis	Optional adapter for cross-instance Socket.IO broadcasts and ephemeral presence support.	REDIS_URL
Cloudinary	Attachment storage and secure delivery URLs.	Server-side upload stream

End-to-end user flow

- 1. The user registers or signs in. The API returns a short-lived access token and a refresh token; the client stores tokens in secure storage.
- 2. The home provider loads workspaces and conversations. A user can create another workspace or join one with a code; the UI refreshes providers immediately after the mutation.
- 3. The user selects a workspace channel or chooses a member to start a direct conversation. Direct-conversation creation is constrained to members of the same workspace.
- 4. The chat provider loads cursor-paginated history from MongoDB, then opens an authenticated Socket.IO connection and joins the authorized conversation room.
- 5. Outgoing messages receive a client-generated ID and appear optimistically. The server persists the message and acknowledges the same client ID with the permanent message ID; duplicate sends are reconciled.
- 6. Socket events deliver new, edited, deleted, typing, presence, and read-state changes to authorized participants. HTTP remains the source for history after restart.
- 7. Attachments are checked by MIME type and 10 MB size, streamed to Cloudinary, and rendered in the chat with an image preview or document metadata.

Realtime event contract

Direction	Events	Behavior
Client -> server	conversation:join / leave	Authorize access, then add or remove the socket from the conversation room.

Direction	Events	Behavior
Client -> server	message:send	Persist text, reply, and attachment metadata; acknowledge with permanent ID plus client ID.
Client -> server	typing:start / stop	Broadcast transient typing state to other authorized room members.
Client -> server	receipt:read	Persist per-user last-read message and broadcast receipt:update.
Server -> client	message:new / updated / deleted	Synchronize persisted message lifecycle across participants.
Server -> client	typing:update / presence:update	Render ephemeral collaboration state.
Server -> client	operation:error	Structured failure for an operation such as unauthorized send.

4. Project structure

Repository layout

```

SyncUp/
|-- lib/
|   |-- models/           Conversation, Message, Workspace
|   |-- providers/        Riverpod application state
|   |-- services/         Dio API client and Socket.IO service
|   |-- screens/          auth, home, workspace, chat
|   |-- theme/            Material 3 SyncUp theme
|   |-- app.dart, main.dart
|-- assets/branding/       SyncUp logo asset
|-- backend/
|   |-- src/server.ts      Express API, Mongoose models, sockets, uploads
|   |-- src/server.test.ts Vitest/Supertest backend checks
|   |-- src/seed.ts        Local demo account seeding
|   |-- package.json, tsconfig.json
|-- Dockerfile             Railway production image
|-- docker-compose.yml     Local MongoDB + Redis
|-- pubspec.yaml           Flutter dependencies
|-- README.md              Local setup and realtime contract

```

Frontend responsibilities

- AuthProvider owns the access/refresh lifecycle and restores a session on app launch.
- Workspace and conversation providers expose loading, empty, error, and refresh states to the home screen.
- ChatScreen renders messages, reply/edit/delete actions, typing state, upload progress, image previews, details, and retry behavior.
- SocketService reconnects with the current access token and surfaces realtime events to the message provider.
- The API client retries once through refresh-token rotation after a 401, then clears the session if refresh fails.

Backend responsibilities

- server.ts defines the request pipeline, validation, Mongoose schemas/indexes, REST routes, upload handling, and Socket.IO handlers.
- Authorization is checked both on REST routes and before joining or using a Socket.IO conversation room.
- The service binds to 0.0.0.0 for Railway/container networking and reads all secrets/configuration from environment variables.

5. Data model, security, and persistence

MongoDB collections

Collection	Important fields / indexes
User	displayName, email, passwordHash, avatarUrl; unique email.
RefreshSession	Hashed token, expiry, revocation timestamp; user index for rotation/revocation.
Workspace / WorkspaceMember	Owner, hashed join code plus last four display digits; unique workspaceId + userId membership.
Channel / ChannelMember	Public/private visibility, archive timestamp; unique channelId + userId membership.
Conversation	Direct or channel kind, workspace/channel references, sorted participantKey; unique direct pair per workspace.
Message	Sender, clientId, text, attachments, replyTo, editedAt, deletedAt; conversationId + createdAt descending index.
ReadState	Per-user lastReadMessageId and readAt; unique conversationId + userId.

Security controls

- Passwords are hashed with bcryptjs; access JWTs expire after 15 minutes and refresh tokens after 30 days.
- Refresh tokens are stored as hashes, rotated on use, and revoked on logout or reuse failure.
- Helmet, CORS allow-listing, JSON size limits, rate limiting, Zod request validation, and MIME/size upload checks are enabled.
- Workspace roles distinguish owner/admin/member; private channels require explicit channel membership.
- Socket handshakes require a valid access token, and each conversation operation re-checks authorization.
- Cloudinary and database credentials are environment variables; no secret values are included in the repository report.

Where messages are stored

Message text and attachment metadata are persisted in MongoDB in the Message collection. The client first displays a local optimistic item, then replaces/reconciles it using the server acknowledgement. On a new session, the app requests history through the cursor-paginated messages endpoint, so messages remain available after refresh, reconnect, or app restart. Attachment binaries are stored by Cloudinary and only their validated URL and metadata are persisted in MongoDB.

6. Public interfaces and deployment

REST interface

Area	Endpoints
Health	GET /api/v1/health
Auth	POST register, login, refresh, logout; GET me
Workspaces	GET/POST workspaces; POST join; rotate-code; members and role management
Channels	List/create/update/archive; join/leave; private member management
Conversations	List/open direct; details; cursor-paginated messages
Messages	PATCH edit; DELETE soft delete
Uploads	POST multipart /api/v1/uploads with validated attachment metadata response

Railway deployment verification

Check	Result
Service	syncup-api
Public URL	https://syncup-api-production.up.railway.app
Latest deployment	SUCCESS; Dockerfile builder; commit c06a250 (Use Railway API by default)
Health probe	GET /api/v1/health returned HTTP 200 and {status: ok}
Runtime	Node service binds to 0.0.0.0; Railway deployment is not sleeping
Configured integrations	MongoDB Atlas persistence; Cloudinary attachment storage; Redis adapter support

Yes - the deployed backend is working

The public health check passed while this report was generated. A successful health response confirms the Railway process is reachable and responding. Application operations still require valid credentials and configured MongoDB/Cloudinary environment variables, which are kept private.

7. Testing and acceptance status

Verification	Status / coverage
Backend tests	Vitest/Supertest checks passed for protected access, registration validation, and localhost CORS handling.
Backend build	TypeScript compilation passed; production start uses dist/server.js.
Flutter tests	Flutter widget/unit test suite passed for the implemented client baseline.
Flutter analysis	Static analysis passed for the current app source.
Deployment	Railway deployment SUCCESS and live health endpoint HTTP 200.
Automated E2E	Not claimed as complete: two-client realtime, production upload, and full restart acceptance remain manual video-demo checks.

Recommended working-video sequence

- Register or sign in as user one, create a workspace, and show the workspace name and join code in details.
- Sign in as user two, join with the code, and show both real member display names in the member list.
- Open a direct chat by selecting the member, type while the other user watches the typing indicator, then send a message.
- Open the workspace channel, send a message, demonstrate read state, quoted reply, edit, and soft delete.
- Upload an image and show the in-chat preview; optionally show an office/PDF attachment card.
- Refresh or restart the app and show that the conversation history is retrieved from the backend.

Known operational notes

- For Android Emulator local development, use http://10.0.2.2:4000/api/v1; the release/default client points to the Railway API.
- Native Android/iOS clients do not rely on browser CORS, while Flutter Web requires the deployed origin to be accepted by the backend CORS policy.
- Redis is optional for a single instance and becomes important when multiple Railway replicas broadcast Socket.IO events.

8. Appendix: setup and handoff

Local backend

```
docker compose up -d
cd backend
cp .env.example .env
npm install
npm run dev
```

Local Flutter client

```
flutter pub get
flutter run
```

```
# Override the API when needed
flutter run --dart-define=API_URL=https://syncup-api-production.up.railway.app/api/v1
```

Delivery summary

SyncUp now has a documented Flutter client, a TypeScript/Express realtime backend, durable MongoDB history, optional Redis scaling, Cloudinary attachments, and a verified Railway deployment. The accompanying working video can demonstrate the acceptance path above using two signed-in users.

Prepared for project submission - SyncUp